ATTESTA

Agent-Readable UI/UX Design Rules

A practical implementation specification for a premium, fintech-inspired security operations dashboard.

VISUAL DIRECTION Premium fintech SaaS	**PRODUCT DOMAIN** CI/CD security and trust
PRIMARY USERS Developers, DevSecOps, security teams	**IMPLEMENTATION MODE** Improve the existing website incrementally

Design principle: Make security decisions feel clear, explainable, and operationally useful. Use the reference for visual direction—not as a screen-by-screen template to copy.

Prepared for coding agents such as Claude Code. Follow this document together with the repository's existing architecture, design tokens, and actual product requirements.

1. How the agent must use this document

This is an implementation contract, not a mood board. Inspect the current project first, then apply the rules without breaking existing behavior.

- **Inspect before editing.** Identify the framework, routes, page components, shared layout, CSS strategy, icon library, chart library, data contracts, and existing design tokens.
- **Preserve product behavior.** Do not remove working routes, API calls, authentication, filters, table actions, or security workflows to make the UI easier to style.
- **Build shared primitives first.** Establish tokens, app shell, sidebar, top bar, page header, cards, badges, tables, filters, empty states, and loading states before polishing individual pages.
- **Implement page by page.** Use the page-specific requirements in Section 8. Keep shared components consistent while allowing each page to have a distinct information hierarchy.
- **Use real data contracts.** Do not invent backend endpoints or claim fabricated security events are real. If data is unavailable, use an explicitly labeled demo/placeholder state.
- **Verify every change.** Run the project's available lint, typecheck, tests, and build. Fix regressions introduced by the UI work.

Agent instruction to follow

“Treat this PDF as the visual and interaction specification. First inspect the repository and report the existing structure. Then implement the design system and shared shell, followed by the requested page. Reuse existing data and behavior. Do not rewrite unrelated architecture. Validate the page at desktop, tablet, and mobile widths.”

Reference boundary

The supplied Behance link is the aesthetic reference. The PDF describes a transferable visual language—premium fintech polish, disciplined spacing, clear metrics, and restrained accents—adapted to Attesta's security use case. Do not reproduce proprietary artwork, exact screens, logos, or unique layouts pixel-for-pixel.

2. Visual direction and design principles

Target feeling

A mature, trustworthy, high-signal B2B SaaS product. It should feel like a carefully designed financial operations console, but the information must communicate pipeline integrity, identity, trust decisions, incidents, and auditability—not personal finance.

Principles

- **Calm confidence:** strong typography, stable alignment, clean surfaces, and minimal decoration.
- **Information before ornament:** every card, chart, badge, and divider must help a user understand state or take action.
- **Density with structure:** display enough security detail for operators, but group it into scannable regions.
- **Explainability:** a trust decision must expose its reason, evidence, policy, and resulting action—not just a red or green label.
- **Consistent meaning:** colors, status labels, icons, and interaction patterns must mean the same thing on all pages.
- **Realistic product UI:** prioritize tables, timelines, event details, policy summaries, and operational metrics over oversized marketing-style hero sections.

Avoid

Generic AI gradients everywhere; excessive glassmorphism; glowing borders; huge empty hero areas; random floating blobs; excessive rounded pills; every item in a separate card; decorative charts without a question to answer; and a different palette on every page.

3. Design tokens: colors, type, spacing, shape

Use CSS variables or the project's existing token mechanism. The values below are a coherent starting point; inspect the reference and current app, then tune them consistently. Do not scatter raw color values throughout components.

Token	Suggested value	Usage
--app-bg	#F4F6FA	Main canvas, if using light mode
--surface	#FFFFFF	Cards, menus, table containers
--surface-muted	#F7F8FB	Subtle nested areas and hover backgrounds
--text-primary	#111827	Titles, primary data, important labels
--text-secondary	#5B6474	Supporting copy, timestamps, metadata
--border	#D9DEE7	Quiet dividers and card outlines
--brand	#3559E0	Primary actions, selected navigation, links
--success	#16805D	Verified/healthy states, only when semantically true
--warning	#B7791F	Warnings, expiring identity, degraded state
--danger	#B42318	Blocked, failed, revoked, confirmed critical issue
--info	#2563A6	Informational states and neutral technical hints

Dark-mode option

If the reference or existing product uses a dark dashboard, use a deep neutral background, slightly elevated surfaces, and accessible text contrast. Do not mix light and dark surfaces randomly. Support one coherent theme first; add a theme toggle only if it is an actual product requirement.

Typography

- Use one highly readable sans-serif family already installed in the project (or a single well-supported web font). Avoid mixing many font families.
- Suggested scale: page title 24–30 px; section heading 16–20 px; card metric 24–32 px; body 14–16 px; table and metadata 12–14 px. Do not make essential data tiny.
- Use medium/semibold weights for hierarchy. Reserve bold for key values and headings, not every label.
- Use tabular numerals for aligned metrics, durations, counts, and timestamps when supported.

Spacing and shape

Use a 4 px base spacing system. Common gaps: 4, 8, 12, 16, 24, 32 px. Typical card padding: 16–24 px. Use consistent 8–12 px card radii and 6–10 px control radii. Avoid mixing many corner-radius styles.

4. Application shell and navigation

Desktop layout

Use a stable left sidebar, a compact top utility bar, and a scrollable main workspace. The content should align to a consistent grid; do not center every page in a narrow marketing container.

Sidebar rules

- Keep the Attesta wordmark/product mark and a concise product descriptor at the top.
- Group navigation into clear sections where useful: Monitor, Investigate, Governance, and System. Keep the group labels subtle.
- Use icon + label for each destination. Icons must be from one icon set and have consistent size/stroke.
- Active navigation must be unmistakable using a quiet tinted background, text/icon emphasis, and optionally a narrow accent indicator.
- Include clear hover, keyboard-focus, disabled, and collapsed states. Do not rely on color alone.
- Keep the user/workspace menu and settings in a predictable lower area. Do not crowd the sidebar with decorative widgets.
- If the sidebar collapses, preserve tooltips or accessible labels. On small screens, convert it into a drawer or suitable mobile navigation.

Top bar rules

- Include page context or breadcrumbs where they help orientation.
- Reserve the right side for relevant utilities such as global search, time range, notifications, help, and profile/workspace controls.
- Show environment/repository context when it materially affects security decisions.
- Avoid adding search, date filters, or notification controls that do not work.

Main content order

Page title + short explanation → primary action(s) → high-priority status or metric row → main analysis/table → secondary detail. The first viewport should answer “where am I?”, “what matters now?”, and “what can I do next?”.

5. Dashboard composition and reusable components

Use a deliberate grid, not a wall of identical cards. Establish hierarchy by varying component size and information density while keeping alignment and spacing consistent.

Component	Rules
Page header	Title, one-sentence purpose, optional breadcrumb, and one primary action. Secondary actions belong in a menu or lower emphasis.
Metric card	Short label, clear value, optional trend/delta with time context, and a small supporting caption. Avoid decorative icons that compete with the value.
Chart panel	Meaningful title, time range, labeled axes/units, useful tooltip, legend only when needed, and a clear empty state.
Data table	Readable column headers, aligned values, row hover/focus, sorting/filtering when useful, stable status labels, and pagination for large datasets.
Status badge	Short text plus restrained semantic color. Pair icons/text with color; do not communicate severity through color alone.
Timeline	Chronological order, timestamps, actor/source, event, outcome, and a way to inspect evidence/details.
Detail drawer/panel	Open from a row when useful. Keep key identity and decision visible; show metadata, evidence, policy, and next actions in sections.
Empty state	Explain why there is no data, whether that is expected, and the next useful action. No generic “Nothing here” alone.
Loading state	Use skeletons that resemble final layout or a concise progress state. Avoid shifting the page when data arrives.
Error state	State what failed, what data may be stale, and a safe retry or recovery action. Do not hide errors behind empty charts.

Card discipline

Use a card when it groups a coherent unit of information. Prefer one well-structured panel with section dividers over five nested cards. Keep shadows subtle or omit them; use borders and background contrast for structure.

6. Security-specific UX rules

Attesta is a security decision product. UI clarity must never weaken the accuracy of identity, evidence, or policy state.

Trust decisions

- Represent the decision explicitly: Allowed, Denied, Review Required, or Indeterminate—use the actual domain vocabulary in the implementation.
- Show the reason next to the decision: policy rule, identity status, evidence check, provenance/attestation result, or failed condition.
- Distinguish “not checked” from “passed”. Missing data must never appear as healthy.
- Show event time, relevant actor/workload, repository, workflow/run, commit or artifact reference, and correlation ID when available.
- For a denial, make the next safe step clear: inspect evidence, view policy, retry after correction, or request authorized review. Never make an unsafe bypass the most prominent action.

Severity and color

State	Visual treatment	Important rule
Healthy / Verified	Muted green label/icon	Only use when verification actually succeeded.
Warning / Expiring	Amber label/icon	Explain what is at risk and when action is needed.
Denied / Failed	Red label/icon	Include reason and evidence; do not make the whole screen alarm-red.
Pending / Review	Blue or neutral label	Make ownership and next action clear.
Unknown / Stale	Neutral label + clear text	Never silently convert unknown or stale into success.

Auditability

Preserve event order, source, timestamp, identity, policy version, evidence references, and decision outcome where the data model provides them. Use copyable IDs and timestamps. Avoid ambiguous labels like “secure” when the UI can say exactly what was verified.

Risky actions

Require confirmation for destructive or high-impact actions. Explain the consequence before the user confirms. Keep revoke, disable, delete, override, and policy-change actions visually distinct from routine navigation. Respect existing authorization and backend enforcement.

7. Interaction, accessibility, responsive behavior

Interaction states

- Every interactive control needs default, hover, focus-visible, active, disabled, loading, and error states where applicable.
- Use buttons for actions and links for navigation. Do not make static cards look clickable.
- Filters should visibly show active values and provide a clear reset. Preserve filters during pagination where practical.
- Use confirmation dialogs for destructive actions, with explicit cancel and confirm labels.
- Use toasts for brief outcomes, but keep important security outcomes visible in the page or event history.
- Do not add animation that delays investigation. Motion should be subtle, brief, and respect reduced-motion preferences.

Accessibility

- Maintain readable contrast for text, icons, borders, focus indicators, and status labels.
- All controls must be keyboard reachable and have visible focus. Provide accessible names for icon-only controls.
- Use semantic headings in order, labels for inputs, table headers for tabular data, and appropriate ARIA only when native semantics are insufficient.
- Never rely on color alone for pass/fail, severity, or selected navigation.
- Make charts understandable through labels, summaries, tooltips, or an equivalent data table.

Responsive behavior

Viewport	Expected behavior
Large desktop (≥1440 px)	Use the full sidebar and multi-column dashboard grid. Avoid stretching cards excessively wide.
Laptop (1024–1439 px)	Preserve primary navigation; reduce grid columns and maintain useful table widths.
Tablet (768–1023 px)	Collapse or compact sidebar; stack secondary panels; allow intentional table overflow.
Mobile (<768 px)	Use drawer/bottom navigation if suitable, stack metrics, simplify chart layout, and prioritize core actions. Do not shrink desktop into unreadable text.

8. Page-by-page requirements for Attesta

Use these as page acceptance criteria. Adapt labels to the routes that already exist in the repository; do not invent new product capabilities just to fill a screen.

01. Overview

Give an at-a-glance security posture. Prioritize trust decisions, active/recent pipeline runs, blocked or suspicious activity, incident count, and service health. Include trends only when the time window and source are clear. Provide direct links from summary cards to filtered detail views.

02. Pipeline Runs

Show GitHub Actions runs with repository/workflow, branch or commit, start time/duration, current state, and final outcome. Support practical filters and sorting. Selecting a run opens its stage timeline and relevant evidence.

03. Run Details / Execution Timeline

Present stages in chronological order: stage name, status, duration, timestamps, runner/workload identity, and linked events. Distinguish failed, skipped, pending, and passed stages. Make the failed stage and its reason easy to find.

04. Trust Decisions

Provide a searchable ledger of allow/deny/review decisions. Show decision, subject/workload, resource, policy, timestamp, and reason. Filters should include outcome, repository, time, and relevant identity dimensions.

05. Identity & Workloads

Explain which machine/workload identity acted, its issuer/source, subject, lifecycle state, and relevant attributes. Separate identity facts from trust conclusions. Do not display secrets or private key material.

06. SPIRE / Identity Service

Show service health, registration/identity status, recent failures, and useful operational metadata only if exposed by the backend. Clearly distinguish service unavailability from an identity verification failure.

07. Policy Management

List policies with name, purpose, scope, version, last updated, and status. Show policy detail and change history. Editing/publishing requires clear validation, confirmation, and appropriate authorization.

08. Evidence & Attestations

Make evidence discoverable by run, artifact, identity, and time. Display provenance/attestation status, issuer, subject, digest/reference, verification result, and reason where available. Never imply cryptographic validity without a successful check.

09. Incidents & Alerts

Prioritize unresolved/high-severity items. Show severity, status, affected repository/run/identity, first/last seen, owner, and next action. Include clear filtering, triage states, and incident detail history.

10. Hyperledger Fabric / Audit Ledger

Show ledger/service health, transaction or event references, timestamps, and verification status when available. Distinguish a recorded event from an independently validated claim. Provide copyable identifiers and drill-down details.

11. Integrations & Repositories

Show connected repositories/providers, connection state, last sync/verification, configured scope, and safe connection/reconnection actions. Make stale connections visible; never show credentials in plaintext.

12. Settings, Users & System Health

Group workspace/profile preferences, access controls, audit settings, and service health logically according to existing product scope. Separate user preferences from privileged system configuration. Require confirmation for sensitive changes.

If the actual 12 route names differ

Map each route to the closest responsibility above. Keep the current routes and naming unless the user explicitly asks for information-architecture changes. If a route is not implemented, do not fabricate a functional page—create a consistent, honest empty/coming-soon state only when appropriate.

9. Tables, charts, forms, and data handling

Tables

- Choose columns based on operator decisions. Put identity/run/event name first, status close to it, then time and secondary metadata.
- Use consistent date/time formatting and timezone context. Keep IDs monospaced or otherwise visually distinct when it improves scanning.
- Avoid truncating critical reasons or statuses. Use a detail panel, expandable row, or tooltip for long supporting values.
- Make sorting direction, active filters, pagination, and selected row state clear.
- For no results, distinguish “no events in this time range” from “failed to load events”.

Charts

- Every chart must answer a specific question, such as trust decisions over time or run outcomes by status.
- Include a clear title, time range, units, axis labels where necessary, and a useful tooltip.
- Avoid 3D charts, unnecessary gradients, misleading truncated axes, and unexplained percentages.
- Provide a data summary or table alternative when charts contain important security information.

Forms and feedback

- Group related fields; use persistent labels rather than placeholder-only labels.
- Validate inline and explain how to fix errors. Preserve user input after validation failures.
- Use explicit save/cancel actions and indicate unsaved changes where relevant.
- For configuration changes, show the scope and likely impact before applying.

Data integrity rule

Never fabricate live metrics, incidents, audit records, or verification results. If mock data is needed for a local prototype, label it “Demo data” in the interface and keep it separate from production data paths.

10. Frontend implementation guidance for coding agents

Adapt to the current stack. Do not force a new library or rewrite the app without a clear need.

Recommended implementation sequence

- **Audit:** inspect package.json, routes, page files, global styles, reusable components, API clients, and tests.
- **Tokens:** add a small, centralized token layer for colors, spacing, typography, radii, shadows, and breakpoints.
- **Shell:** implement the sidebar, top bar, content container, page header, and responsive navigation.
- **Primitives:** implement buttons, status badges, cards, table styles, filters, dialogs, skeletons, empty/error states, and typography helpers.
- **One representative page:** polish Overview as the visual baseline, then reuse the system on the remaining routes.
- **Page mapping:** update one page at a time, preserving its data and actions. Avoid a giant rewrite across all 12 pages.
- **Quality pass:** test breakpoints, keyboard focus, empty/error/loading states, long IDs, long repository names, and large tables.

Component and code quality

- Use existing conventions and naming. Keep components small and reusable, but do not create abstractions for one-off decoration.
- Keep UI state separate from data fetching where the current architecture supports it.
- Reuse the project's existing icon and chart libraries. Avoid emoji as production status icons.
- Use design tokens rather than duplicated inline styling. Avoid `!important` patches and page-specific overrides that break shared components.
- Do not add dependencies without checking whether the project already has an equivalent.
- Keep loading, error, empty, and success states connected to actual data/action results.

Suggested token snippet

```
:root {
--app-bg: #F4F6FA;
--surface: #FFFFFF;
--text-primary: #111827;
--text-secondary: #5B6474;
--border: #D9DEE7;
--brand: #3559E0;
--radius-card: 12px;
--space-1: 4px;
--space-2: 8px;
--space-3: 12px;
--space-4: 16px;
--space-6: 24px;
}
```

11. Visual QA and acceptance checklist

The agent must verify the following before calling a page complete.

- ■ Page uses the shared shell, spacing, typography, colors, and component styles.
- ■ Page title and purpose are clear without reading the entire screen.
- ■ Primary action is obvious; secondary actions do not compete.
- ■ Security state is explicit and never relies on color alone.
- ■ Every displayed metric has a trustworthy source, clear time range, or explicit demo label.
- ■ Tables remain readable with long names, IDs, timestamps, and empty results.
- ■ Charts have clear labels and meaningful empty/error states.
- ■ Buttons, filters, menus, dialogs, pagination, and row actions actually work.
- ■ Loading, empty, error, success, disabled, hover, and keyboard-focus states are covered as relevant.
- ■ No destructive action is triggered accidentally or made visually indistinguishable from routine actions.
- ■ No secrets, tokens, private keys, or sensitive environment values are exposed in the UI.
- ■ Desktop, tablet, and mobile layouts have been inspected.
- ■ No horizontal overflow appears except intentional table scrolling.
- ■ Existing routes, API behavior, and authentication still work.
- ■ Lint/typecheck/tests/build pass, or any pre-existing failures are clearly reported.

Definition of done

A page is complete only when it is visually consistent, responsive, accessible, connected to real behavior, and validated—not merely when it resembles the reference screenshot.

12. Copy-paste master prompt for the coding agent

Use this prompt alongside the PDF. For page-by-page work, replace [PAGE NAME] with the specific route and avoid asking the agent to rebuild all pages at once.

```
You are improving the existing Attesta CI/CD security dashboard. Follow the attached “Attesta
Agent-Readable UI/UX Design Rules” PDF as the design specification.

VISUAL DIRECTION
Use a premium, polished fintech-SaaS visual language inspired by the supplied Behance
reference: strong information hierarchy, disciplined spacing, refined typography, clean
surfaces, restrained color, and professional data visualization. Adapt it to security
operations. Do not copy the reference pixel-for-pixel, and do not turn this into a marketing
landing page.

FIRST, INSPECT
Inspect the current repository, framework, routes, design system, shared components, API/data
contracts, and existing behavior. Identify what already exists before changing code. Do not
replace the stack or rewrite unrelated architecture.

IMPLEMENT
1. Reuse or establish centralized design tokens.
2. Reuse the shared sidebar, top bar, page header, cards, status badges, tables, filters,
dialogs, and loading/empty/error states.
3. Keep security decisions explainable: show state, reason, evidence, timestamp, and related
identity/run/policy when data exists.
4. Preserve all existing functionality, routes, authentication, and data fetching.
5. Never fabricate production security data. Clearly label demo data.
6. Make controls functional and accessible. Include keyboard focus and responsive behavior.
7. Keep code consistent with the existing project and avoid unnecessary dependencies.

CURRENT TASK
Implement or refine only this page: [PAGE NAME].
Use the PDF’s page-specific requirements. Make this page visually consistent with the shared
design system, not a one-off redesign. Do not modify unrelated pages unless a shared component
needs a compatible update.

VERIFY
Run available lint, typecheck, tests, and build. Inspect desktop, tablet, and mobile layouts.
Check loading, empty, error, and long-content states. Report what files changed, what behavior
was preserved, tests run, and any limitations. Do not claim checks passed unless you actually
ran them.
```

Reference

Behance reference supplied by the user: AI CFO Platform — Fintech SaaS Dashboard & Investor App UI. The exact page may change or be inaccessible; the rules here define the intended transferable design language.

End of specification